Microstates and Macroscopic Thermodynamics of a Binary System

Statistical Physics — Laboratory Practice 1

Group X

Rocío Fernández Jover (NIA: 100523129, GitHub: `@user1`)
J. Cantillana (NIA: 100496704, GitHub: `@jcantillana04`)
Olivia (NIA: 100XXXXXX, GitHub: `@olivia614`)
Author 4 (NIA: 100XXXXXX, GitHub: `@user4`)

Department of Physics, Universidad Carlos III de Madrid

October 29, 2026

Abstract

We study a system of N independent unbiased coins as an exact model of a two-state assembly. A modular Python package enumerates all 2^N microstates by bitwise encoding, computes the multiplicities $\Omega(N,k)$ exactly and in log-space, and evaluates probabilities and Boltzmann entropies. Full enumeration is limited by its $\mathcal{O}(2^N)$ cost: the largest system fully enumerated within five minutes of wall-clock time is $N = 36$. For the assigned system of $N = 117$ coins the equal-split macrostate has probability 0.07330, while 0.9473 of all outcomes lie within ± 10 heads of it. Entropy differences between macrostates are of order $10\,k_{\mathrm{B}}$, i.e. 10^{-22} J/K. As $N \to 10^{23}$ the relative width of the distribution falls to 3.16×10^{-12}, and macroscopic decreases of entropy become unobservable. This is the statistical content of the Second Law.

Repository: https://github.com/olivia614/StatisticalPhysicsLab1

1 Introduction & Objectives

A microstate of N distinguishable coins is the ordered list of the face of every coin. A macrostate is specified only by the number of heads k. Every one of the 2^N microstates is equally likely for unbiased coins. The multiplicity of a macrostate is the number of ways to choose which k coins show heads:

$$\Omega(N,k) = \binom{N}{k} = \frac{N!}{k!\,(N-k)!}, \qquad \sum_{k=0}^{N} \Omega(N,k) = (1+1)^N = 2^N. \tag{1}$$

The probability of a macrostate and its Boltzmann entropy are therefore

$$P(N,k) = \frac{\Omega(N,k)}{2^N}, \qquad S(N,k) = k_{\mathrm{B}} \ln \Omega(N,k). \tag{2}$$

The objectives are: (i) to enumerate microstates and determine how far brute force can go; (ii) to compute and plot $\Omega(N,k)$ for $N = 20$–100; (iii) to evaluate probabilities and entropy changes for $N = 100 + M$; and (iv) to extrapolate to $N \sim 10^{23}$ and interpret the Second Law statistically.

2 Methodology & Collaborative Workflow

2.1 Environment & Reproducibility

The environment is a Conda environment (`statphys_lab1`, Python 3.11, NumPy, SciPy, Matplotlib, pytest) specified in `environment.yml`. Running `python main.py` regenerates every figure in `report/figures/` and every number in this report (`report/results.tex`). The timing study runs with `python main.py -benchmark`.

2.2 Version Control & Repository Architecture

Development followed a feature-branch workflow on GitHub, with direct pushes to `main` disabled and pull requests reviewed by at least one teammate. The code is organised as:

- `src/microstates.py`: bitwise enumeration (generator and chunked NumPy versions), benchmark utilities.
- `src/combinatorics.py`: exact big-integer and log-gamma binomial coefficients, floating-point limits.
- `src/thermodynamics.py`: probabilities, entropy, waiting times, Gaussian limit, Monte Carlo tosses.
- `tests/`: 68 `pytest` unit tests checking every routine against `math.comb` and analytical identities.
- `main.py`: orchestrates all calculations; `report/`: LaTeX source and figures.

2.3 Numerical uncertainty

All combinatorial quantities are exact integers or are computed in double precision with relative error below 10^{-12}; this is verified by unit tests against exact integer arithmetic. They are quoted to three or four significant figures. The only measured quantities are wall-clock times. These are reported as the mean $\pm$ sample standard deviation of three repetitions on a 16-thread laptop CPU.

3 Computational Analysis of Microstates

3.1 Enumeration for $N = 7$

Each microstate is encoded as an integer $s \in [0, 2^N)$ whose bit i is 1 if coin i shows heads. Iterating over `range(1 << N)` with a Python generator visits every microstate exactly once in $\mathcal{O}(1)$ memory. The macrostate is the population count of s (`s.bit_count()`). Table 1 summarises the 128 outcomes for $N = 7$; the complete listing is given in Appendix C. The counts reproduce row 7 of Pascal's triangle and sum to 2^7.

Table 1: Macrostates of $N = 7$ coins: multiplicity $\Omega(7, k)$ obtained by enumeration, and probability P_k.

Heads k	**Tails** $7-k$	**Example microstate**	$\mathbf{\Omega(7,k)}$	$\mathbf{P_k}$
7	0	HHHHHHH	1	$1/128 = 0.0078$
6	1	THHHHHH	7	$7/128 = 0.0547$
5	2	TTHHHHH	21	$21/128 = 0.1641$
4	3	TTTHHHH	35	$35/128 = 0.2734$
3	4	TTTTHHH	35	$35/128 = 0.2734$
2	5	TTTTTHH	21	$21/128 = 0.1641$
1	6	TTTTTTH	7	$7/128 = 0.0547$
0	7	TTTTTTT	1	$1/128 = 0.0078$
		Total	128	1

3.2 Largest N for full enumeration

Two implementations were timed (Table 2, Fig. 1). The first is the pure-Python generator. The second splits $[0, 2^N)$ into blocks of 2^{22} consecutive integers; for each block it computes popcounts in compiled NumPy code (`np.bitwise_count`) and distributes the blocks over 16 processes. Both genuinely visit every microstate; no combinatorial shortcut is used. A least-squares fit of $\log_2 t = a + bN$ over the four largest N of each method (smaller N are dominated by fixed overheads) gives slopes $b = 1.045$ (generator) and $b = 0.920$ (vectorised), to be compared with $b = 1$ for a doubling of the cost per added coin. Because the timings are noisy over a short range of N, the extrapolation instead imposes the theoretical scaling $t = c\,2^N$. The cost per microstate c is taken as the median of $t/2^N$, giving $c = 307.1\,\text{ns}$ (generator) and $c = 3.83\,\text{ns}$ (vectorised, wall-clock over all processes). Extrapolating to $t = 300\,\text{s}$ gives $N_{\max} \approx 29.9$ for the generator and $N_{\max} \approx 36.2$ for the parallel version. The extrapolation was checked with direct runs (stars in Fig. 1). The largest N enumerated within the limit is $N_{\max} = 36$, which took 99 s; its histogram reproduced $\Omega(N, \lfloor N/2 \rfloor)$ exactly. Timings depend on the hardware and on background load, with a run-to-run scatter of about 5–15%.

Table 2: Wall-clock time (mean ± s.d., 3 runs) for building the full histogram of 2^N microstates. Memory is the peak traced allocation per process (generator / one vectorised worker); the last rows are single verification runs.

N	$\mathbf{2^N}$	**Generator (s)**	**Vectorised (s)**	**Mem. gen / vec (MB)**
16	6.55×10^4	0.0101 ± 0.00034	—	0.01 / —
18	2.62×10^5	0.0437 ± 0.0079	—	0.01 / —
20	1.05×10^6	0.195 ± 0.025	—	0.01 / —
22	4.19×10^6	1.39 ± 0.12	—	0.01 / —
24	1.68×10^7	5.28 ± 0.31	—	0.01 / —
26	6.71×10^7	22.5 ± 0.34	0.847 ± 1.1	0.01 / 100.00
28	2.68×10^8	—	2.02 ± 0.048	— / 100.00
30	1.07×10^9	—	7.36 ± 0.052	— / 100.00
32	4.29×10^9	—	12.2 ± 7.2	— / 100.00
34	1.72×10^{10}	—	82.6 ± 18	— / 100.00
36	6.87×10^{10}	—	99 (1 run)	—

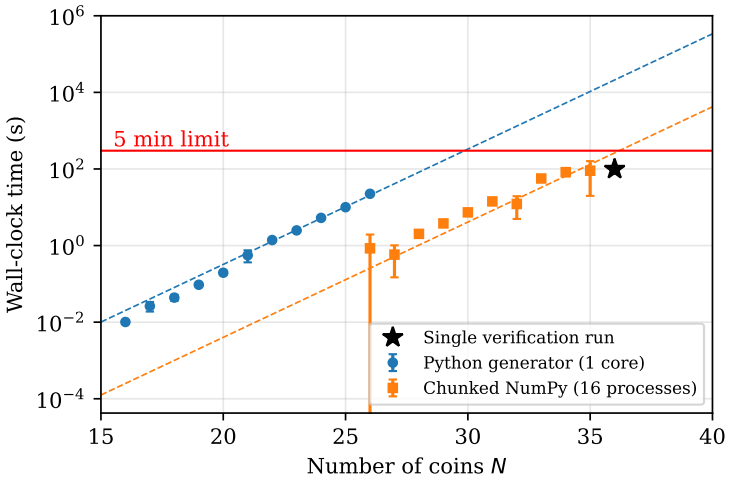


Figure 1: Enumeration time versus N (log scale). Dashed lines are the exponential fits; the horizontal line is the five-minute limit.

Why larger N cannot be reached. The limit is the exponential size of the state space, not the implementation. Each additional coin doubles the number of microstates and hence the run time. At ~ 1 ns per state even an ideal code needs 2^{40} ns ≈ 18 min for $N = 40$ and about 36 years for $N = 60$. Storing the states instead of generating them would need 8×2^N bytes, i.e. 8 GB at $N = 30$ and 8 TB at $N = 40$; generators avoid this. Writing the list (as required for $N = 7$) costs $N \cdot 2^N$ characters, which is already 32 GB at $N = 30$. Fixed-width integer encoding also caps the vectorised method at $N = 63$.

3.3 Macrostate distributions ($N = 20, 50, 80, 100$)

For these N, listing microstates is impossible, so $\Omega(N, k)$ is evaluated directly. Factorials are never formed: the code uses exact Python integers via the Pascal recurrence $\Omega(N, k) = \Omega(N - 1, k - 1) + \Omega(N - 1, k)$ for the plots, and log-space evaluation

$$\ln \Omega(N, k) = \ln \Gamma(N + 1) - \ln \Gamma(k + 1) - \ln \Gamma(N - k + 1) \tag{3}$$

for probabilities. Figure 2 shows the results. The peak multiplicity grows from 1.848×10^5 ($N = 20$) to 1.009×10^{29} ($N = 100$), while the peak probability falls from 0.1762 to 0.0796. The distribution broadens in k as $\sqrt{N}$, but it narrows relative to N.

As a check, 100 000 simulated tosses of $N = 100$ coins (Appendix B, Fig. 5) give mean $\langle k \rangle = 49.991 \pm 0.016$ (expected 50). Comparing the simulated histogram with Eq. (2) gives $\chi^2 = 41.3$ for 42 degrees of freedom, so the two agree within counting uncertainty.

Maximum N that can be calculated. There is no hard limit with the methods above; only the representation limits it.

- In IEEE double precision, the total 2^N overflows ($> 1.80 \times 10^{308}$) at $N = 1024$.
- A naive $N!/[k!(N - k)!]$ already fails at $N = 171$.
- The largest multiplicity $\Omega(N, N/2)$ overflows at $N = 1030$.

Arbitrary-precision integers remove the overflow, at a cost that grows with the number of digits ($\approx 0.3N$). Eq. (3) has no limit at all: the unit tests evaluate it at $N = 10^{23}$, where it agrees with Stirling's formula to 10^{-12}.

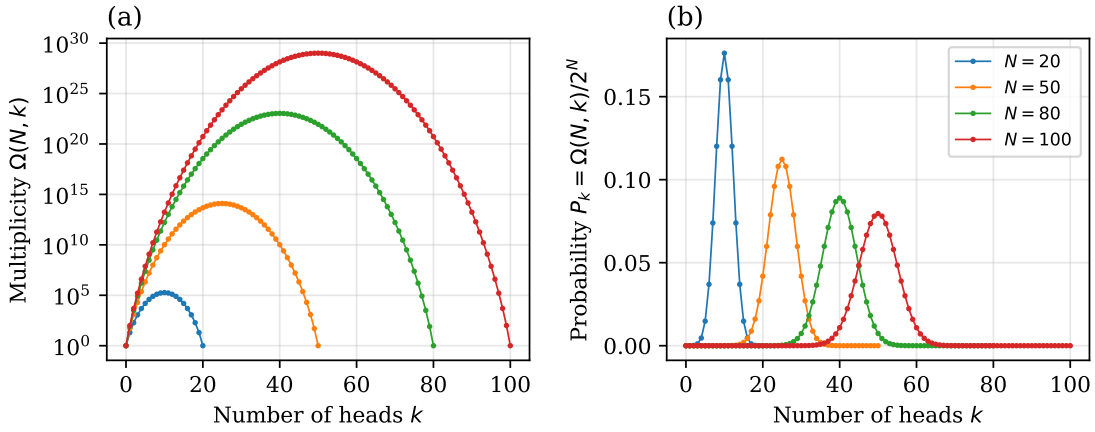


Figure 2: (a) Number of microstates $\Omega(N,k)$ with k heads (log scale). (b) Normalised probability $P_k = \Omega(N,k)/2^N$.

3.4 Thermodynamic limit ($N \to 10^{23}$)

Applying Stirling's formula $\ln N! \approx N\ln N - N + \frac{1}{2}\ln(2\pi N)$ to Eq. (1) and expanding around $k = N/2$ to second order in $\delta = k - N/2$ gives the de Moivre–Laplace result

$$P(N,k) \simeq \sqrt{\frac{2}{\pi N}}\exp\left[-\frac{2(k-N/2)^2}{N}\right], \qquad \mu = \frac{N}{2}, \quad \sigma = \frac{\sqrt{N}}{2}, \quad \frac{\sigma}{\mu} = \frac{1}{\sqrt{N}}. \tag{4}$$

The approximation is already accurate to 0.25% at the peak for $N = 100$. For $N = 10^{23}$ the absolute width $\sigma = 1.58 \times 10^{11}$ is large, but the relative width is $\sigma/\mu = 3.16 \times 10^{-12}$ (Fig. 3).

- The fraction of heads is $1/2$ to eleven significant figures.
- A deviation $|k/N - \frac{1}{2}| > 1 \times 10^{-10}$ is a 63σ event, with probability 3.25×10^{-871}.
- Even a 0.1% deviation ($k/N = 0.501$) is suppressed by a factor $10^{-8.69\times10^{16}}$.

The distribution is then effectively a delta function: the system is always found in the most probable macrostate, which is the equilibrium state.

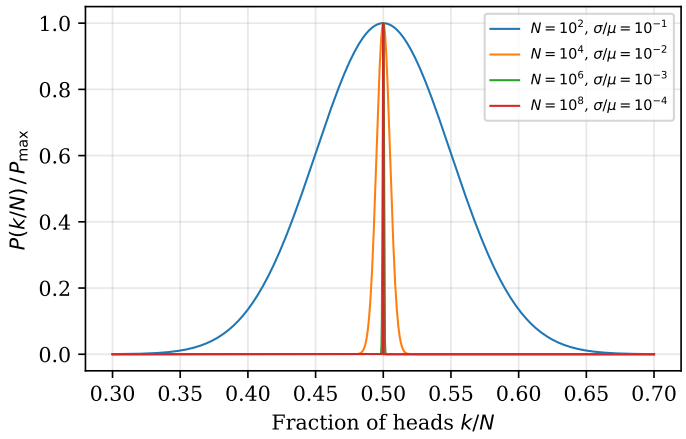


Figure 3: Probability distribution versus the fraction of heads k/N for increasing N (Eq. 4). The relative width scales as $N^{-1/2}$.

4 Probabilities, Entropy, and Thermodynamics

4.1 Parameter assignment

M is the arithmetic mean of the last two digits of the team members' NIAs $(29, 04)$, which is 16.50, rounded to $M = 17$. Hence $N = 100 + M = 117$ coins, with $2^N = 1.662 \times 10^{35}$ microstates. Since N is odd, "$N/2$ heads" is taken as $\lfloor N/2 \rfloor = 58$.

4.2 Probability and entropy analysis

All probabilities follow from Eq. (2). Range probabilities are sums over k, evaluated with a log-sum-exp to avoid underflow. Entropy changes for a transition $k_1 \to k_2$ are

$$\frac{\Delta S}{k_{\mathrm{B}}} = \ln \frac{\Omega(N, k_2)}{\Omega(N, k_1)}, \qquad k_{\mathrm{B}} = 1.380649 \times 10^{-23}\ \mathrm{J/K}. \tag{5}$$

Results are collected in Table 3 and illustrated in Fig. 4.

Table 3: Probabilities and entropy changes for $N = 117$ coins (questions 1–9).

Question	Quantity	Result
1	$P(k = 30)$, with $\Omega = 7.101 \times 10^{27}$	4.27×10^{-8}
2	$P(k = 58)$, with $\Omega = 1.218 \times 10^{34}$	0.07330
3	$P(58 - 10 \le k \le 58 + 10)$ heads	0.9473
4	$P(58 - 20 \le N - k \le 58 + 20)$ tails	0.999862
5	ΔS: $k = 83 \to 58$ heads (Ω ratio 3.57×10^4)	$10.484\, k_{\mathrm{B}} = 1.45 \times 10^{-22}$ J/K
6	ΔS: $k = 68 \to 58$ heads (Ω ratio 4.63)	$1.532\, k_{\mathrm{B}} = 2.12 \times 10^{-23}$ J/K
7	ΔS: $k = 58 \to 20$ heads (Ω ratio 1.39×10^{-12})	$-27.300\, k_{\mathrm{B}} = -3.77 \times 10^{-22}$ J/K
8	$P(k = 8)$; mean wait at 1 toss/s	4.10×10^{-24}; 2.44×10^{23} s
9	$P(58)/P(M + 10 = 27)$	4.96×10^7
	$P(58)/P(M - 10 = 7)$	2.46×10^{23}

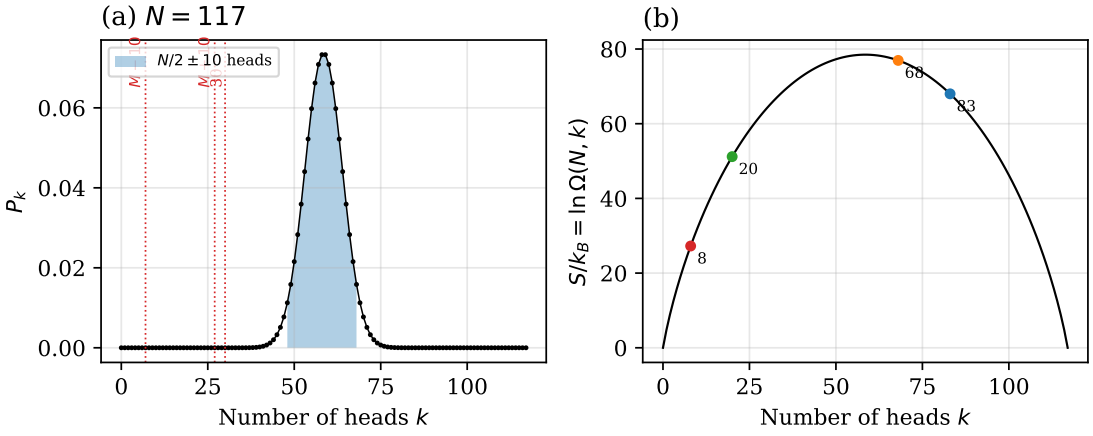


Figure 4: (a) P_k for $N = 117$; the shaded band is 58 ± 10 heads and dotted lines mark $k = 30$ and $M \pm 10$. (b) Entropy $S/k_{\mathrm{B}} = \ln \Omega$; points mark the states used in questions 5–8.

Discussion of questions 1–4. Although 58 heads is the single most probable outcome, its probability is only 0.07330 because there are $117 + 1$ possible macrostates. Ranges matter more:

about $\sigma = \sqrt{N}/2 \approx 5.4$, so ± 10 heads covers $\approx 1.8\sigma$ and holds 0.9473 of the probability. The ± 20-tails window ($\approx 3.7\sigma$) misses only 1.38×10^{-4}. By contrast, $k = 30$ lies 5.3σ below the mean and has probability 4.27×10^{-8}.

Question 8. Each toss is an independent trial with success probability $p = P(N, 8)$. The number of tosses until the first success is therefore geometric, with mean $1/p$. At one toss per second the expected wait is 2.44×10^{23} s, which is 7.72×10^{15} years or 5.60×10^{5} times the age of the Universe.

Question 9. The equal split is 4.96×10^{7} times more likely than $M + 10 = 27$ heads, and 2.46×10^{23} times more likely than $M - 10 = 7$ heads.

4.3 Second Law of Thermodynamics interpretation (question 10)

None of these results contradicts the Second Law, because in statistical mechanics the Second Law is a statement about probabilities, not a strict prohibition. The microscopic dynamics (re-tossing coins) are reversible, and every microstate is equally likely, including the "ordered" ones. A transition from $k = 58$ to $k = 20$ lowers the entropy by $-27.300\, k_{\mathrm{B}}$. It is not forbidden; it is simply $\Omega(58)/\Omega(20) = e^{27.3}$ times rarer than the reverse. For $N = 117$, fluctuations of a few k_{B} (questions 5–6) do occur: about 5.3% of tosses fall outside 58 ± 10. These fluctuations are visible only because N is small. The Second Law is the statement that the macrostate with maximum Ω (maximum S) overwhelmingly dominates, so a spontaneous evolution almost always goes towards higher entropy.

4.4 The limit $N \gg 1$ (question 11)

For $N \sim 10^{23}$, Eq. (4) and the log-space routines give:

- **Fixed k (30 heads, 8 heads, $M \pm 10$).** The probability is $\sim 2^{-N}$. For example, $P(k = 8) = 10^{-3.01 \times 10^{22}}$, so the waiting time and the likelihood ratios of question 9 become $10^{\sim 3 \times 10^{22}}$. These events never happen.
- $P(N/2) = \sqrt{2/\pi N} = 2.52 \times 10^{-12} \to 0$, and so does any window of fixed width ± 10 or ± 20: that width is negligible compared with $\sigma = 1.58 \times 10^{11}$. What tends to 1 is the probability of any *relative* window $|k/N - \frac{1}{2}| < \varepsilon$.
- **Entropy.** Fixed offsets ($N/2 + 25 \to N/2$) give $\Delta S \simeq 2\delta^2 k_{\mathrm{B}}/N \approx 10^{-20} k_{\mathrm{B}} \to 0$. Macroscopic offsets ($k/N = 0.501 \to 0.5$) give $\Delta S = -2.00 \times 10^{17}\, k_{\mathrm{B}}$ in magnitude, i.e. about -2.76×10^{-6} J/K, which is a measurable thermodynamic entropy.

In the thermodynamic limit, therefore, the probabilistic statement of question 10 becomes a deterministic law. Entropy-decreasing macroscopic fluctuations have probabilities $\exp(-\Delta S/k_{\mathrm{B}}) \sim 10^{-10^{17}}$, and the Second Law holds without exception in practice.

5 Conclusion

Bitwise enumeration confirms $\Omega(N, k) = \binom{N}{k}$ directly, but the 2^N cost limits it to $N = 36$ in five minutes, even with vectorisation and parallelism. Log-space combinatorics removes all overflow limits and allows exact statements for $N = 117$ and asymptotic ones for $N = 10^{23}$. The binomial distribution narrows relative to N as $N^{-1/2}$. Entropy-lowering fluctuations, observable for $N \sim 10^2$, become unobservable for macroscopic N. This is the statistical origin of the Second Law of Thermodynamics.

A AI Collaboration & Audit Statement

Following Phase 3 of the course guidelines, AI assistance (Anthropic Claude Code, model Claude Opus 5.5) was used for planning, code generation and drafting this report. Every prompt, the files generated and the team's modifications are logged in `AI_STATEMENT.md` at the repository root. The planning transcript is in `docs/ai_plans/`. Verification consisted of:

1. 68 `pytest` unit tests comparing every AI-written routine with exact integer arithmetic (`math.comb`), identities ($\sum_k \Omega = 2^N$, symmetry) and published values ($\Omega(100, 50) \approx 1.009 \times 10^{29}$).
2. An independent Monte Carlo cross-check of the distribution ($\chi^2/\mathrm{dof} = 41.3/42$).
3. Re-derivation by the team of all formulas in Sections 3–4.

Issues found during the audit are documented in `AI_STATEMENT.md`: a floating-point edge case in the overflow-limit routine, and a LAPACK/MKL crash in the Conda environment.

B Monte Carlo cross-check

Tossing $N = 100$ coins 100 000 times with NumPy's pseudo-random generator (seed 2026) reproduces the exact distribution within the Poisson counting uncertainty $\sqrt{n_k}$ of each bin.

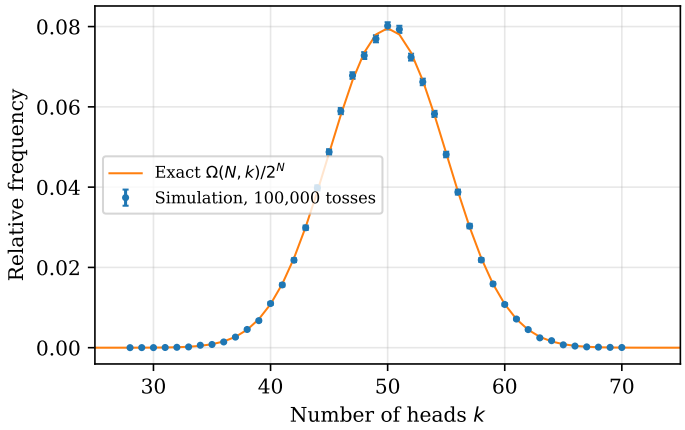


Figure 5: Monte Carlo relative frequencies for $N = 100$ (error bars: Poisson $\sqrt{n_k}$) against the exact distribution.

C Complete list of microstates for $N = 7$

Macrostate (Ω)	Microstates
7H 0T (1)	HHHHHHH
6H 1T (7)	THHHHHH, HTHHHHH, HHTHHHH, HHHTHHH, HHHHTHH, HHHHHTH, HHHHHHT
5H 2T (21)	TTHHHHH, THTHHHH, THHTHHH, THHHTHH, THHHHTH, THHHHHT, HTTHHHH, HTHTHHH, HTHHTHH, HTHHHTH, HTHHHHT, HHTTHHH, HHTHTHH, HHTHHTH, HHTHHHT, HHHTTHH, HHHTHTH, HHHTHHT, HHHHTTH, HHHHTHT, HHHHHTT
4H 3T (35)	TTTHHHH, TTHTHHH, TTHHTHH, TTHHHTH, TTHHHHT, THTTHHH, THTHTHH, THTHHTH, THTHHHT, THHTTHH, THHTHTH, THHTHHT, THHHTTH, THHHTHT, THHHHTT, HTTTHHH, HTTHTHH, HTTHHTH, HTTHHHT, HTHTTHH, HTHTHTH, HTHTHHT, HTHHTTH, HTHHTHT, HTHHHTT, HHTTTHH, HHTTHTH, HHTTHHT, HHTHTTH, HHTHTHT, HHTHHTT, HHHTTTH, HHHTTHT, HHHTHTT, HHHHTTT
3H 4T (35)	TTTTHHH, TTTHTHH, TTTHHTH, TTTHHHT, TTHTTHH, TTHTHTH, TTHTHHT, TTHHTTH, TTHHTHT, TTHHHTT, THTTTHH, THTTHTH, THTTHHT, THTHTTH, THTHTHT, THTHHTT, THHTTTH, THHTTHT, THHTHTT, THHHTTT, HTTTTHH, HTTTHTH, HTTTHHT, HTTHTTH, HTTHTHT, HTTHHTT, HTHTTTH, HTHTTHT, HTHTHTT, HTHHTTT, HHTTTTH, HHTTTHT, HHTTHTT, HHTHTTT, HHHTTTT
2H 5T (21)	TTTTTHH, TTTTHTH, TTTTHHT, TTTHTTH, TTTHTHT, TTTHHTT, TTHTTTH, TTHTTHT, TTHTHTT, TTHHTTT, THTTTTH, THTTTHT, THTTHTT, THTHTTT, THHTTTT, HTTTTTH, HTTTTHT, HTTTHTT, HTTHTTT, HTHTTTT, HHTTTTT
1H 6T (7)	TTTTTTH, TTTTTHT, TTTTHTT, TTTHTTT, TTHTTTT, THTTTTT, HTTTTTT
0H 7T (1)	TTTTTTT

D Core algorithms

```
def iter_microstates(n):                # src/microstates.py
    state, total = 0, 1 << n            # 2**n microstates, O(1) memory
    while state < total:
        yield state
        state += 1

counts[state.bit_count()] += 1          # macrostate = number of set bits

# chunked version: popcount a block of 2**22 states in compiled code
states = offsets + np.uint64(block << chunk_bits)
counts += np.bincount(np.bitwise_count(states), minlength=n + 1)

def log_binom(n, k):                    # src/combinatorics.py, Eq. (3)
    return gammaln(n + 1) - gammaln(k + 1) - gammaln(n - k + 1)
```